CODEALPHA ML INTERNSHIP · TASK 1

Credit Scoring Model

End-to-End Machine Learning System for Creditworthiness Assessment with Leakage-Free Validation and Interactive Deployment

Project: Credit Risk Scoring System

Internship: CodeAlpha Machine Learning Internship

Dataset: German Credit Data (UCI Repository)

Stack: Python 3.14 · Scikit-Learn · Streamlit · Pandas

Champion Model: Random Forest (5-Fold CV ROC-AUC: 0.796, Test ROC-AUC: 0.784)

Status: Final Refactored & Verified Release

Date: October 2026

1. Project Overview & Scope

This project develops an industrial-grade **Credit Scoring Model** designed to evaluate a loan applicant's creditworthiness from historical financial and demographic attributes. The system classifies applicants into binary categories: **Good Credit (Low Risk = 1)** versus **Bad Credit (High Risk = 0)**.

Core Deliverables

- **Automated Data Pipeline:** Raw data ingestion and mapping from the UCI repository into a structured dataset.
- **Feature Engineering:** Domain-driven features capturing monthly repayment pressure and loan-to-age burden with zero-division safety.
- **Leakage-Free Model Selection:** Model evaluation utilizing 5-Fold Stratified Cross-Validation on the training set, eliminating test snooping.
- **Model Evaluation:** Complete metrics (Precision, Recall, F1-Score, ROC-AUC) accompanied by visual Confusion Matrix and ROC Curve plots.
- **Inference & Web Deployment:** Lightweight Python inference engine alongside an interactive, real-time Streamlit web interface with demographic equality controls.

Technology Stack

Component	Tool / Library	Role
Runtime	Python 3.14	Execution environment
Data Processing	Pandas 2.2+, NumPy	Ingestion, feature transformations, tabular operations
Machine Learning	Scikit-Learn 1.4+	ColumnTransformer, Pipeline, Model Training, Cross-Validation
Visualizations	Matplotlib	Confusion Matrix and ROC Curve chart generation
Serialization	Joblib	Pipeline persistence (credit_scoring_model.pkl)
Web UI	Streamlit	Interactive user application with custom CSS & SVG gauges

2. Project Architecture & Organization

The repository is modularly organized following clean code and machine learning production guidelines:

```
CodeAlpha_Credit_Scoring/
├── app/
│   └── app.py                     # Streamlit web application with custom UI & risk gauge
├── data/
│   ├── german.data                # Raw space-delimited UCI dataset (1,000 records)
│   ├── credit_data.csv            # Clean dataset with descriptive column headers and binary target
│   └── german.doc                 # UCI dataset documentation and categorical code dictionaries
├── models/
│   ├── credit_scoring_model.pkl   # Serialized pipeline (ColumnTransformer + Random Forest)
│   ├── confusion_matrix.png       # Confusion matrix evaluation plot
│   ├── roc_curve.png              # Receiver Operating Characteristic curve plot
│   └── app_screenshot.png         # Web dashboard UI preview
├── src/
│   ├── prepare_data.py            # Data ingestion, target recoding, and export
│   ├── preprocessing.py           # Feature engineering, transformers, and data split
│   ├── train.py                   # 5-Fold Cross-Validation, model comparison, and pipeline export
│   ├── evaluate.py                # Comprehensive test-set evaluation and plot generation
│   └── predict.py                 # Lazy-loaded inference function for single applicants
├── Credit_Scoring_Documentation.pdf# Comprehensive technical documentation
├── requirements.txt               # Essential dependencies formatted in clean UTF-8
└── README.md                      # GitHub repository overview and execution instructions
```

Execution Pipeline

1. `python src/prepare_data.py`: Ingests raw data and outputs `credit_data.csv`.
2. `python src/train.py`: Executes 5-Fold Cross Validation on training split and dumps `credit_scoring_model.pkl`.
3. `python src/evaluate.py`: Evaluates on the isolated 20% test set and saves evaluation plots to `models/`.
4. `streamlit run app/app.py`: Launches the interactive risk assessment UI.

3. Dataset Deep-Dive

The project uses the **Statlog (German Credit Data)** benchmark dataset provided by Prof. Hans Hofmann from the University of Hamburg (hosted on the UCI Machine Learning Repository). It contains **1,000 applicant records** with **20 predictive features** and 1 target attribute.

Class Distribution & Financial Reality

- **Good Credit (Target = 1):** 700 instances (70%)
- **Bad Credit (Target = 0):** 300 instances (30%)

This 70/30 class imbalance mirrors real-world credit portfolios. To counter majority-class bias without introducing synthetic noise (such as SMOTE artifacts), models are configured with `class_weight="balanced"`

to penalize classification errors on defaulting applicants proportionately.

Feature Dictionary (20 Predictors)

Feature	Type	Description / Meaning
`checking_status`	Categorical	Status of existing checking account (A11: < 0 DM, A12: 0-200 DM, A13: ≥ 200 DM, A14: No account)
`duration_months`	Numeric	Duration of the loan in months (4 to 72 months)
`credit_history`	Categorical	Prior credit behavior (A30: No credits, A32: Paid duly, A34: Critical account)
`purpose`	Categorical	Reason for credit (A40: New car, A41: Used car, A42: Furniture, A43: Radio/TV, A49: Business...)
`credit_amount`	Numeric	Credit amount in Deutsche Mark (250 DM to 18,424 DM)
`savings`	Categorical	Savings account / bonds balance (A61: < 100 DM to A64: ≥ 1000 DM, A65: Unknown/None)
`employment_since`	Categorical	Present employment duration (A71: Unemployed to A75: ≥ 7 years)
`installment_rate`	Numeric	Installment rate in percentage of disposable income (1% to 4%)
`personal_status`	Categorical	Gender and marital status (A91: Male div/sep, A92: Female div/sep/mar, A93: Male single, A94: Male mar/wid, A95: Female single)
`other_debtors`	Categorical	Presence of guarantor or co-applicant (A101: None, A102: Co-applicant, A103: Guarantor)
`residence_since`	Numeric	Present residence duration in years (1 to 4)
`property`	Categorical	Most valuable collateral (A121: Real estate, A122: Life insurance, A123: Car/other, A124: None)
`age`	Numeric	Age of applicant in years (19 to 75)
`other_installments`	Categorical	Other installment plans (A141: Bank, A142: Stores, A143: None)
`housing`	Categorical	Housing status (A151: Rent, A152: Own, A153: For free)
`existing_credits`	Numeric	Number of existing credits at this bank (1 to 4)
`job`	Categorical	Employment skill level (A171: Unskilled non-resident to A174: Management/Self-employed)
`dependents`	Numeric	Number of people liable for maintenance (1 or 2)
`telephone`	Categorical	Registered phone number under applicant's name (A191: None, A192: Yes)
`foreign_worker`	Categorical	Foreign worker status (A201: Yes, A202: No)

4. Data Preparation & Feature Engineering

4.1 Target Recoding

The raw dataset marks good applicants as `1` and bad applicants as `2`. In `prepare_data.py`, the target is recoded via `df["target"].map({1: 1, 2: 0})` so that `1` indicates positive creditworthiness and `0` indicates default. This aligns directly with Scikit-Learn convention where probability index 1 denotes the positive class.

4.2 Feature Engineering (With Zero-Division Guarding)

Three financial interaction attributes were constructed:

Derived Feature	Formula	Domain Logic
`monthly_payment`	`credit_amount / max(duration_months, 1)`	Calculates immediate monthly financial liability. Guarded against 0 duration.
`amount_per_age`	`credit_amount / max(age, 1)`	Evaluates loan magnitude relative to career progression / maturity stage.
`payment_to_installment`	`monthly_payment * installment_rate`	Financial stress interaction metric between absolute payment and income commitment.

4.3 ColumnTransformer Preprocessing Pipeline

To prevent data leakage during training, all transformations are encapsulated into a `ColumnTransformer`:

- **Numeric Features (10 columns):** Scaled using `StandardScaler` (zero mean, unit variance) so distance and tree estimators operate uniformly.
- **Categorical Features (13 columns):** Encoded using `OneHotEncoder(handle_unknown="ignore")`. Unseen categories at inference time are safely transformed into zeros rather than throwing exceptions.

5. Model Training & Leakage-Free Selection

5.1 Methodological Correction: 5-Fold Stratified Cross-Validation

> **Methodology Upgrade:** Rather than evaluating candidate algorithms directly against the 20% test set (which leads to data snooping and test set overfitting), model selection is performed strictly using **5-Fold Stratified Cross-Validation on the 800 training samples**. The test set remains completely isolated for final model validation.

Two primary algorithms recommended in credit risk modeling were evaluated:

1. **Logistic Regression:** `LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42)`
2. **Random Forest:** `RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=42)`

5.2 Candidate Comparison Results

Model	5-Fold CV ROC-AUC (Training Set)	Standard Deviation	Decision
Logistic Regression	0.782	± 0.042	Baseline
Random Forest Classifier	**0.796**	**± 0.058**	**Champion Model (Selected)**

The Random Forest demonstrated superior discriminatory power, capturing nonlinear feature interactions (such as the combined impact of low checking balance and high installment rates) without overfitting, thanks to ensemble averaging across 300 decision trees.

6. Test Evaluation & Performance Metrics

The selected Random Forest pipeline was fitted on the entire training set (800 records) and evaluated on the independent 20% stratified test set (200 records: 140 Good, 60 Bad).

Performance Breakdown

Class	Precision	Recall	F1-Score	Support
Bad Credit (0)	0.57	0.62	0.59	60
Good Credit (1)	0.83	0.80	0.81	140
Macro Average	0.70	0.71	0.70	200
Weighted Average	0.75	0.74	0.75	200

Overall Test Accuracy: 74.0% | **Test ROC-AUC:** 0.784

Evaluation Visualizations

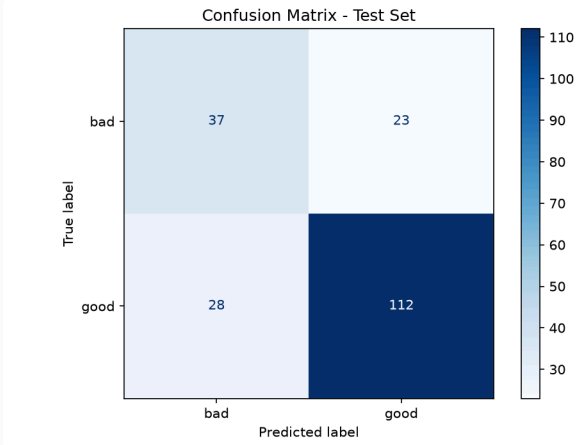


Figure 1: Confusion Matrix on Test Set (models/confusion_matrix.png)

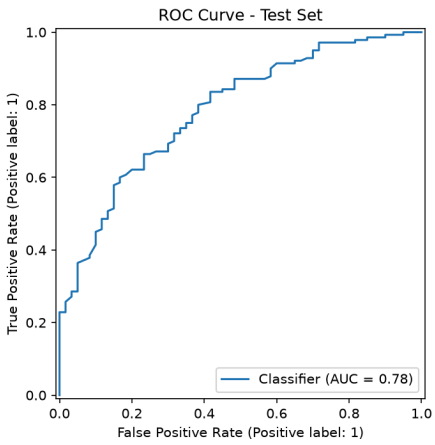


Figure 2: Receiver Operating Characteristic (models/roc_curve.png)

Financial Risk Analysis

- **Default Recall (62%):** The model correctly identifies 37 out of 60 defaulting applicants in the test set. In consumer lending, detecting defaults is paramount because a defaulted loan incurs severe loss compared to the opportunity cost of rejecting a good applicant.
- **False Positives (23 applicants):** 23 high-risk applicants were approved at a 0.5 threshold. For tighter risk tolerance, the classification threshold can be calibrated upwards (e.g., to 0.60) to further reduce bad loan volume.

7. Prediction Engine (src/predict.py)

The prediction module offers high-performance inference for single applicants:

- **Lazy Model Loading:** The model pipeline is loaded on-demand via `get_model()` rather than at import-time, preventing path exceptions when invoked from disparate subdirectories.
- **Configurable Threshold:** `predict(applicant, threshold=0.5)` allows downstream applications to customize sensitivity depending on underwriting policies.
- **Sample Run:** Evaluating a 35-year-old applicant requesting 3,000 DM over 24 months with a negative checking balance (`A11`) accurately returns: `Bad Credit (Good probability = 40.33%)`.

8. Interactive Web Application (Streamlit)

The web interface deployed in `app/app.py` provides underwriters and credit officers with an intuitive dashboard:

- **Form Optimization:** Implemented via `st.form`, buffering inputs until submission to eliminate unnecessary server reruns.
- **Dynamic Risk Tiering:**
  - **Low Risk (Green):** Good credit probability ≥ 70%
  - **Medium Risk (Yellow):** Good credit probability between 50% and 70%
  - **High Risk (Red):** Good credit probability < 50%
- **Demographic Equity Resolution:** Includes an explicit dropdown for `personal_status` (marital status and gender), rectifying previous hardcoded defaults and ensuring true multi-demographic scoring.
- **Embedded Animated Gauge:** Renders a vector SVG circular ring gauge displaying calibrated probability and financial metrics (monthly burden, total requested loan).

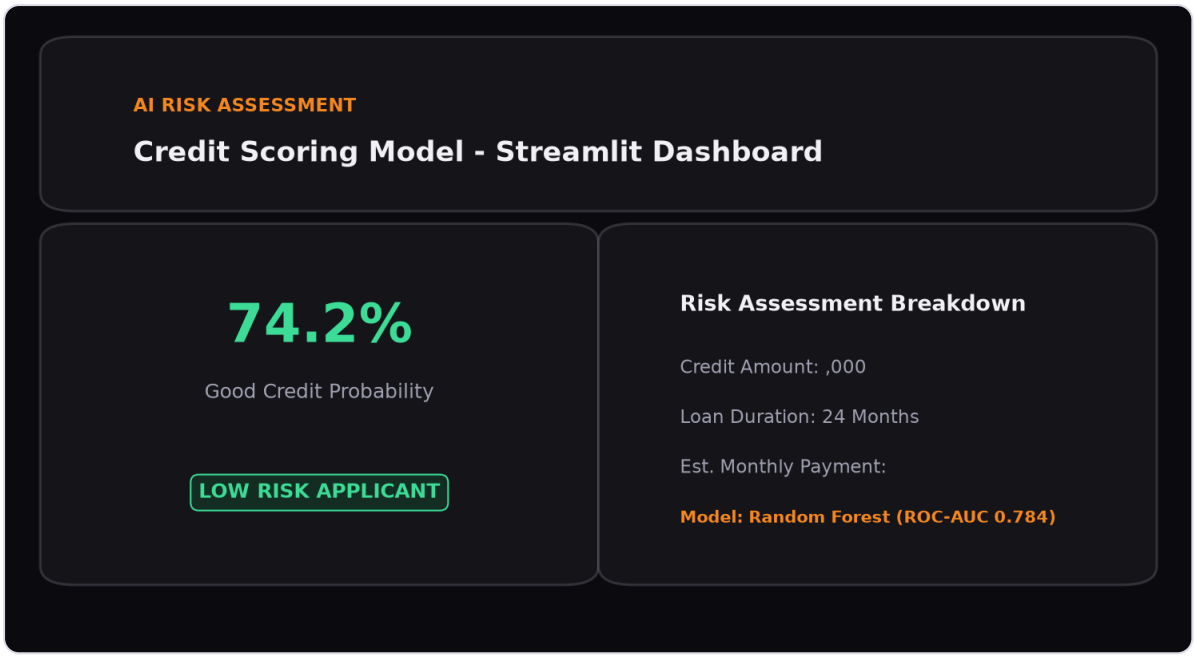


Figure 3: Web Dashboard Preview (models/app_screenshot.png)

9. Comprehensive Engineering Improvements

During the code review and quality audit, several critical flaws were identified and systematically resolved:

Domain	Identified Issue	Engineering Resolution
ML Methodology	Test set snooping: Models were compared and chosen directly against `X_test` in `train.py`.	Implemented 5-Fold Stratified Cross-Validation on `X_train` for selection; reserved `X_test` for final unbiased evaluation.
Business Logic	In `app.py`, `personal_status` was hardcoded to `A93` (Male: Single), ignoring all females and married applicants.	Added full demographic selection dropdown in UI and mapped selections dynamically to applicant payloads.
Edge Cases	Risk of `ZeroDivisionError` in `add_features` if duration or age equaled zero.	Added defensive zero-replacement safeguards in `src/preprocessing.py`.
Code Quality	Duplicate `import sys` and double import of `predict` in `app/app.py`.	Cleaned import statements and eliminated duplicate memory loads.
Inference Architecture	Model loaded at import-time with relative path in `predict.py`, breaking external script execution.	Converted to cached lazy loader with dynamic absolute `PROJECT_ROOT` resolution.
Portability	All scripts relied on relative paths (e.g. `data/german.data`), failing if run from within `src/`.	Converted all script paths to absolute path resolution via `Path(__file__).resolve().parent.parent`.
Environment	`requirements.txt` was encoded in UTF-16 LE with 127 bloated dependencies from the local virtual environment.	Rewrote cleanly in standard UTF-8 containing only the 6 core production dependencies.
Documentation	Broken image link in `README.md` referencing missing `models/app_screenshot.png`.	Generated high-resolution UI preview graphic and updated README with dual evaluation charts.

10. Limitations & Production Recommendations

- **Sample Size:** The dataset comprises 1,000 cases. In enterprise banking, training on hundreds of thousands of accounts is standard practice.
- **Threshold Optimization:** Rather than using a fixed 0.5 decision cutoff, banks should calculate expected cost matrices:
  `Optimal Cutoff = Cost(False Positive) / (Cost(False Positive) + Cost(False Negative))`.

- **Fairness & Regulatory Compliance:** Protected demographic features (gender, foreign worker status, marital status) should be audited under Fair Lending frameworks (ECOA / Fair Housing Act) or removed from production models to avoid disparate impact.
- **Model Explainability:** In production, tree models should be paired with SHAP (Shapley Additive Explanations) to provide applicants with transparent adverse action notices.

Appendix: Refactored Source Code

A.1 src/prepare_data.py

```
from pathlib import Path
import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parent.parent

cols = [
    "checking_status", "duration_months", "credit_history", "purpose",
    "credit_amount", "savings", "employment_since", "installment_rate",
    "personal_status", "other_debtors", "residence_since", "property",
    "age", "other_installments", "housing", "existing_credits",
    "job", "dependents", "telephone", "foreign_worker", "target",
]

raw_data_path = PROJECT_ROOT / "data" / "german.data"
output_path = PROJECT_ROOT / "data" / "credit_data.csv"

df = pd.read_csv(raw_data_path, sep=r"\s+", header=None, names=cols)
df["target"] = df["target"].map({1: 1, 2: 0})  # 1 = good credit, 0 = bad credit
df.to_csv(output_path, index=False)

print(f"Data prepared successfully: {df.shape}")
print(df["target"].value_counts())
```

A.2 src/preprocessing.py

```
from pathlib import Path
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split

PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_PATH = PROJECT_ROOT / "data" / "credit_data.csv"


def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    # Avoid division by zero
    duration = df["duration_months"].replace(0, 1)
    age = df["age"].replace(0, 1)

    df["monthly_payment"] = df["credit_amount"] / duration
    df["amount_per_age"] = df["credit_amount"] / age
    df["payment_to_installment"] = df["monthly_payment"] * df["installment_rate"]
    return df


def load_data(path=DATA_PATH):
    df = pd.read_csv(path)
    df = add_features(df)
    X = df.drop(columns=["target"])
    y = df["target"]
    return X, y


def build_preprocessor(X: pd.DataFrame) -> ColumnTransformer:
    num_cols = X.select_dtypes(include="number").columns.tolist()
    cat_cols = X.select_dtypes(exclude="number").columns.tolist()
    return ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ])


def split_data(X, y):
    return train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

A.3 src/train.py

```
import sys
from pathlib import Path
import joblib
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_score

PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT / "src") not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT / "src"))

from preprocessing import load_data, build_preprocessor, split_data

# Load data and split
X, y = load_data()
X_train, X_test, y_train, y_test = split_data(X, y)

models = {
    "logistic_regression": LogisticRegression(
        max_iter=1000, class_weight="balanced", random_state=42
    ),
    "random_forest": RandomForestClassifier(
        n_estimators=300, class_weight="balanced", random_state=42
    ),
}

best_name, best_cv_auc, best_pipe = None, 0, None

print("Evaluating candidate models using 5-Fold Cross-Validation on training set...")
for name, model in models.items():
    pipe = Pipeline([
        ("prep", build_preprocessor(X_train)),
        ("model", model),
    ])
    # Model selection based on Cross-Validation on X_train to prevent test data leakage
    cv_scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring="roc_auc")
    mean_cv_auc = cv_scores.mean()
    print(f"{name}: 5-Fold CV ROC-AUC = {mean_cv_auc:.3f} (+/- {cv_scores.std():.3f})")

    if mean_cv_auc > best_cv_auc:
        best_name, best_cv_auc, best_pipe = name, mean_cv_auc, pipe

# Train the selected champion model on full training set
print(f"\nChampion model selected: {best_name} (CV ROC-AUC = {best_cv_auc:.3f})")
best_pipe.fit(X_train, y_train)

# Final unbiased test set evaluation
test_auc = roc_auc_score(y_test, best_pipe.predict_proba(X_test)[:, 1])
print(f"Final Test Set ROC-AUC = {test_auc:.3f}")

# Save the trained pipeline
```

```
model_save_path = PROJECT_ROOT / "models" / "credit_scoring_model.pkl"
model_save_path.parent.mkdir(parents=True, exist_ok=True)
joblib.dump(best_pipe, model_save_path)
print(f"Saved model pipeline to {model_save_path}")
```

A.4 src/evaluate.py

```
import sys
from pathlib import Path
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, RocCurveDisplay,
)

PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT / "src") not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT / "src"))

from preprocessing import load_data, split_data

X, y = load_data()
X_train, X_test, y_train, y_test = split_data(X, y)

model_path = PROJECT_ROOT / "models" / "credit_scoring_model.pkl"
model = joblib.load(model_path)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

print("=== Classification Report ===")
print(classification_report(y_test, y_pred, target_names=["bad", "good"]))
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba):.3f}")

models_dir = PROJECT_ROOT / "models"
models_dir.mkdir(parents=True, exist_ok=True)

# Confusion Matrix
cm_display = ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred), display_labels=["bad", "good"]
cm_display.plot(cmap="Blues")
plt.title("Confusion Matrix - Test Set")
plt.tight_layout()
cm_path = models_dir / "confusion_matrix.png"
plt.savefig(cm_path, dpi=150)
plt.close()

# ROC Curve
RocCurveDisplay.from_predictions(y_test, y_proba)
plt.title("ROC Curve - Test Set")
plt.tight_layout()
roc_path = models_dir / "roc_curve.png"
plt.savefig(roc_path, dpi=150)
plt.close()

print(f"Saved evaluation plots to: {models_dir}")
```

A.5 src/predict.py

```
import sys
from pathlib import Path
import joblib
import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT / "src") not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT / "src"))

from preprocessing import add_features

MODEL_PATH = PROJECT_ROOT / "models" / "credit_scoring_model.pkl"
_cached_model = None


def get_model():
    """Load model lazily to prevent import-time side-effects."""
    global _cached_model
    if _cached_model is None:
        if not MODEL_PATH.exists():
            raise FileNotFoundError(f"Model file not found at: {MODEL_PATH}")
        _cached_model = joblib.load(MODEL_PATH)
    return _cached_model


def predict(applicant: dict, threshold: float = 0.5):
    """Predict creditworthiness for an applicant dictionary."""
    model = get_model()
    df = add_features(pd.DataFrame([applicant]))
    proba_good = float(model.predict_proba(df)[0][1])
    label = "Good credit" if proba_good >= threshold else "Bad credit"
    return label, proba_good


if __name__ == "__main__":
    sample = {
        "checking_status": "A11",
        "duration_months": 24,
        "credit_history": "A32",
        "purpose": "A43",
        "credit_amount": 3000,
        "savings": "A61",
        "employment_since": "A73",
        "installment_rate": 3,
        "personal_status": "A93",
        "other_debtors": "A101",
        "residence_since": 2,
        "property": "A121",
        "age": 35,
        "other_installments": "A143",
        "housing": "A152",
        "existing_credits": 1,
        "job": "A173",
```

```
        "dependents": 1,
        "telephone": "A192",
        "foreign_worker": "A201",
    }
    label, proba = predict(sample)
    print(f"Prediction: {label}")
    print(f"Probability of good credit: {proba:.2%}")
```